Amazon CloudFront for CDN

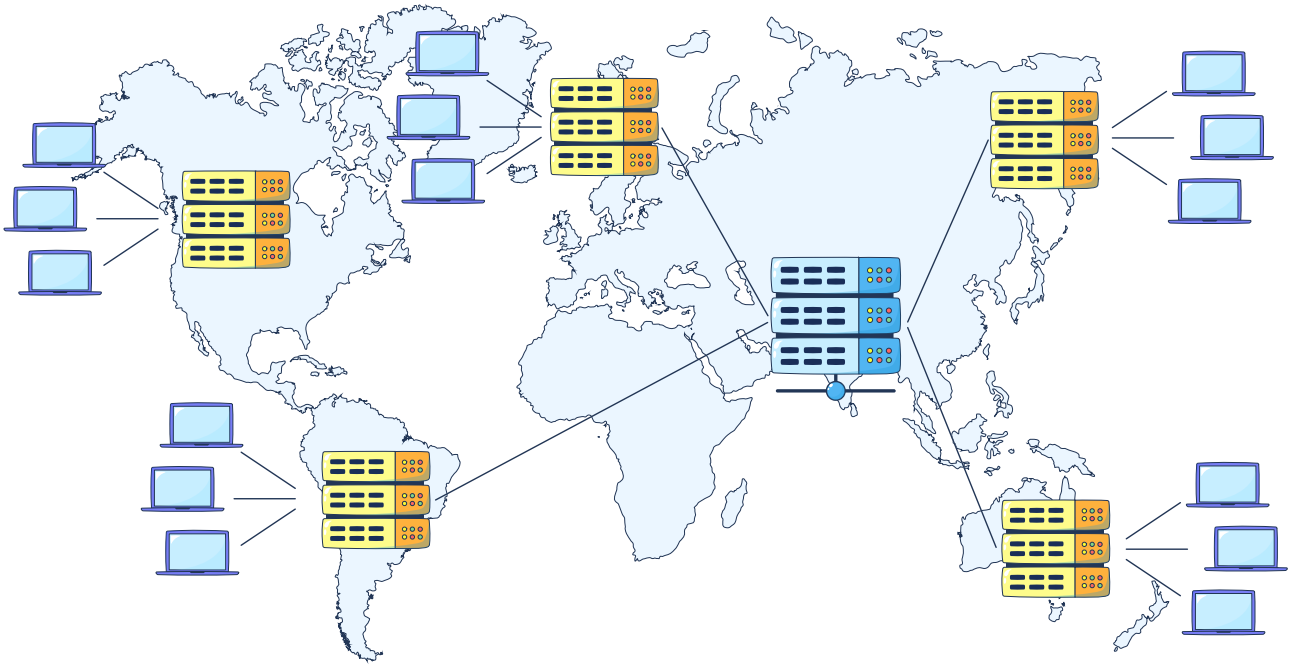
Explore how Amazon CloudFront works as a global content delivery network to speed up web applications. Learn to configure origins, distributions, cache behaviors, and secure content delivery using signed URLs and geo restrictions. Understand how CloudFront integrates with API Gateway for efficient caching and environment isolation.

When users access a web application worldwide, every request that hits the origin server—be it an S3 bucket, EC2 instance, or API Gateway endpoint—adds latency and load.

Imagine a user in Sydney requesting an image hosted in an S3 bucket in Ohio. Each request has to travel across continents and back, introducing noticeable delays. At the same time, the origin server carries the burden of handling repeated requests for the same content, which is inefficient and costly.

This is where content delivery networks (CDNs) step in. A **CDN** is a globally distributed network of *edge locations* that cache and serve content closer to the user's location. By offloading static and dynamic content delivery to the edge, CDNs dramatically improve latency, throughput, and reliability while reducing the operational burden on your origin.

Ask



Amazon CloudFront is AWS's native CDN solution, tightly integrated with other AWS services. However, it isn't just a tool for making websites faster—it's a critical part of modern cloud architectures. Developers use CloudFront to deliver content with low latency, reduce load on origin servers, and implement intelligent caching and security mechanisms at the edge.

Let's explore how CloudFront works and how you can effectively deliver high-performance content from custom origins, S3 buckets, and API Gateway endpoints. We'll also explore how to fine-tune caching behavior and secure content access.

How CloudFront works

Ask

At a high level, Amazon CloudFront is a global network of *edge locations* that work together with the origin and distribution settings to deliver content efficiently.

Think of it as a smart delivery system with regional dispatch centers that know when and how to serve packages—except here, the packages are web content.

Let's break down the core components that make CloudFront function:

1. Origin

Origins serve as the source locations for the content distributed through CloudFront. These can be:

- **Amazon S3 buckets:** Commonly used to store static website content, media files, and software downloads.
- **Custom origins:** External HTTP/HTTPS servers, Elastic Load Balancers (ELBs), or any other web server.

The code snippet given below shows the Python code to create a CloudFront distribution with an S3 origin:

main.py Python

```python
import boto3

client = boto3.client('cloudfront')

response = client.create_distribution(
    DistributionConfig={
        'CallerReference': 'unique-ref-001',
        'Origins': {
            'Items': [{
                'Id': 'S3-origin',
                'DomainName': 'mybucket.s3.amazonaws.com',
                'S3OriginConfig': {
                    'OriginAccessIdentity': ''
```

Ask

Python code to create a CloudFront distribution with S3 origin

2. Distribution

A **distribution** is the CloudFront configuration that tells it how to behave. Each distribution specifies:

- **Origins:** One or more origins from which CloudFront retrieves content.
- **Settings:** Cache behaviors, origin request settings, SSL configurations, and more.
- **Domain name:** A unique domain name assigned to the distribution (e.g., `d1234.cloudfront.net`).

Each distribution has a unique CloudFront domain name (like `d123abcd.cloudfront.net`) that the users or applications will access.

3. Edge locations

Edge locations are globally dispersed data centers where CloudFront caches the content. When a user requests an object, the request is routed to the nearest edge location.

If the edge location already has the content cached, it's served instantly—this is a **cache hit**. Otherwise, it results in a **cache miss**, and CloudFront retrieves the content from the origin and caches it for future use.

Ask

How CloudFront distribution works

Regional edge caches

Regional edge caches are larger than edge locations and are fewer in number. They store less frequently accessed content and help reduce latency for users accessing content from specific geographic regions.

4. Cache behavior

CloudFront's **cache behaviors** determine how specific content paths are cached and served. You can define behaviors for different URL patterns—e.g., cache `/images/*` differently than `/api/*`.

For example, a developer hosts a marketing site on S3 and a dynamic API behind API Gateway. They use CloudFront to cache static assets for 24 hours, but API responses for only 1 minute.

Ask

Key configuration parameters include:

- `PathPattern`: Match specific routes.
- `TTL settings`: To customize how long content stays cached at edge locations, CloudFront allows us to set the value of `MinimumTTL`, `MaximumTTL` and `DefaultTTL`.
- Developers can also use **cache-control headers** from the origin to override or complement CloudFront's TTLs.
- `ForwardedValues`: Decide whether to cache responses that depend on cookies or query strings.

This flexibility helps us fine-tune performance and origin load reduction.

How to secure CloudFront distributions

CloudFront supports signed URLs and signed cookies, allowing developers to serve private content only to authorized users. This is commonly used for paywalled videos or downloadable assets.

For example, to restrict access to training videos hosted on S3, we can use signed URLs that expire in 10 minutes to prevent hotlinking or link sharing. The code snippet given below shows the Python code for this task:

main.py Python

```python
from datetime import datetime, timedelta
from cloudfront_signed_url import generate_signed_url

url = generate_signed_url(
    url="https://d123.cloudfront.net/protected/video.mp4",
    key_pair_id="APKEXAMPLE",
    private_key_file="path/to/private_key.pem",
    expire_time=datetime.utcnow() + timedelta(minutes=10)
)

print("Signed URL:", url)
```

Ask

Python code to secure access to CloudFront distribution using presigned URLs

CloudFront also supports **geographic restrictions**, which can block or allow content based on the user's location. For example, a company might block access to licensing-restricted content from users outside of the United States. This can be done in a few simple steps by adding the countries to the white list of a CloudFront distribution.

Use case: Serving API Gateway with CloudFront

One of the most powerful and practical applications of Amazon CloudFront is using it as a distribution layer in front of API Gateway. This setup gives developers fine-grained control over how API responses are cached, secured, and routed—without burdening the API Gateway or backend services directly.

Imagine a scenario where we're building a multi-tenant web application that fetches data through RESTful APIs exposed via API Gateway. These APIs include both public data (which can be cached) and private data (which should not be). Placing CloudFront in front of API Gateway lets us selectively cache responses, enforce HTTPS, add custom security headers, and even segment environments cleanly using path-based behaviors.

Performance and caching advantages

CloudFront significantly boosts the performance of API Gateway by caching GET requests at edge locations, reducing repeated load on the APIs. For endpoints that return frequently accessed data—like public blog posts, product catalogs, or configuration files—this means much faster response times and lower execution costs.

Ask

CloudFront with API Gateway

We can also configure short TTLs (e.g., 60 seconds) for dynamic but semi-static data, ensuring freshness while still benefiting from edge caching.

TLS termination and security enhancements

CloudFront handles **TLS (HTTPS) termination**, offloading the encryption overhead from API Gateway. This allows us to use a *custom domain with an ACM-issued certificate*, improving branding and trust.

Integrating **AWS WAF (Web Application Firewall)** with the CloudFront distribution adds another layer of protection by allowing us to:

- Block malicious IPs.
- Filter requests based on query strings or headers.
- Mitigate DDoS attacks using rate limiting and geo-matching rules.

Routing dev and prod via path behaviors

Ask

An often-overlooked feature is how CloudFront behaviors can isolate development and production environments using URL path patterns. For example:

- `/api/prod/*` → routes to the production API Gateway stage.
- `/api/dev/*` → routes to the development stage.

When combined with **stage variables** in API Gateway and **Lambda authorizers**, this enables clean environment separation without hard-coding stage names or changing frontend URLs.

> **Tip:** Using CloudFront with API Gateway not only optimizes performance but also simplifies environment management. By defining multiple cache behaviors in our distribution and associating them with different API Gateway stages, we can dynamically switch between dev, test, and prod

Ask